Refinamento Técnico — PrevisaoCerta

Como construir o painel: arquitetura, contratos de dados, API, modelos de código e plano de testes

Desenvolvimento Front-End [React] T4 | Módulo 1 | Semana 11 | Projeto condutor: **Desafio PrevisaoCerta**
Leia antes o **Documento Funcional** (o quê e por quê, com a identidade visual). Aqui está o **como**. Os arquivos completos estão na pasta `modelo-de-arquivos`, ao lado deste documento: **HTML e CSS prontos** (a base visual e semântica) e **JavaScript com funções-chave em branco (`TODO`)** para você completar. Cada `TODO` traz a dica do que fazer.

---

1. Visão técnica

Item	Decisão
Tecnologias	HTML5, CSS3 e JavaScript puros (scripts clássicos, sem módulos, sem bibliotecas)
Como rodar	Abrir a pasta no VS Code e usar o **Live Server** (`http://127.0.0.1:5500`). Necessário para a Geolocation (contexto seguro)
API	Open-Meteo: Geocoding e Forecast. Sem chave, sem cadastro, CORS liberado
Persistência	`localStorage`, com chaves prefixadas por `previsaocerta:`
Padrões	Mobile-first; `textContent` em vez de `innerHTML` para dados de fora; nomes de funções em português; um timer só

2. Estrutura de arquivos

```
previsaocerta/
├── index.html            ← estrutura semântica (landmarks, formulário, card, região aria-live)
├── css/
│   └── style.css         ← tokens da marca, base mobile-first, componentes, media queries
├── js/
│   ├── storage.js        ← "memória": localStorage (cidades, unidade, auto, cache)
│   ├── api.js            ← "internet": fetch, Geocoding, Forecast, decodificarTempo, formatarTemperatura
│   ├── ui.js             ← "tela": status, aviso (setTimeout), renderizarLista, renderizarCard
│   └── app.js            ← "maestro": eventos, fluxo, Geolocation, setInterval, visibilitychange
└── img/
    ├── logo-agrolog.svg  ← logo (decorativo, alt="")
    └── favicon.svg
```

> **Ordem dos scripts importa**
> No `index.html`, os `<script>` entram na ordem `storage.js` → `api.js` → `ui.js` → `app.js`: cada arquivo usa funções do anterior. O `app.js` é o único que "executa" algo no carregamento (chama `iniciar()`).

3. Responsabilidade de cada arquivo (e o que NÃO fazer)

Arquivo	Faz	Não faz
`storage.js`	Ler e gravar no `localStorage`; converter JSON; devolver valores padrão	Não mexe no DOM, não chama `fetch`
`api.js`	Montar URLs, chamar `fetch`, checar `response.ok`, devolver dados limpos	Não mexe no DOM, não usa `localStorage`
`ui.js`	Desenhar na tela a partir de dados recebidos; status e aviso	Não busca dados, não grava no storage
`app.js`	Ligar eventos e coordenar os três acima; timers; geolocalização	Não monta HTML na mão (isso é do `ui.js`)

4. Contratos de dados

4.1 Chaves do localStorage

Chave	Tipo guardado	Exemplo de valor (texto)	Padrão na 1ª visita
`previsaocerta:cidades`	Array de objetos (JSON)	`[{"nome":"Curitiba","latitude":-25.4284,"longitude":-49.2733}]`	`[]` (via `|| []`)
`previsaocerta:unidade`	String	`celsius` ou `fahrenheit`	`'celsius'` (via `||`)
`previsaocerta:auto`	String `'true'` / `'false'`	`true`	`false`
`previsaocerta:cache`	Objeto indexado pelo nome da cidade (JSON)	`{"Curitiba": {"temperatura":22.4,"codigo":3,"vento":9.1,"hora":"2026-10-09T09:30","salvoEm":1790000000000}}`	`{}`

> **Lembrete**
> O `localStorage` só guarda texto. Objetos e arrays passam por `JSON.stringify` ao salvar e `JSON.parse` ao ler. **Nunca** guarde token, senha ou CPF.

4.2 Objetos em memória

```
// Cidade (guardada na lista)
{ nome: 'Curitiba', latitude: -25.4284, longitude: -49.2733 }

// Clima (devolvido por buscarClima e guardado no cache)
{ temperatura: 22.4,        // sempre em °C vindo da API
  codigo: 3,                // weather_code (padrão WMO)
  vento: 9.1,               // km/h
  hora: '2026-10-09T09:30', // hora local da cidade
  salvoEm: 1790000000000 }  // só no cache: Date.now()
```

5. A API Open-Meteo

5.1 Forecast (clima atual)

```
GET https://api.open-meteo.com/v1/forecast
    ?latitude=-25.4284
    &longitude=-49.2733
    &current=temperature_2m,weather_code,wind_speed_10m
    &timezone=auto
```

Resposta (resumida): os dados "agora" ficam em `current`.

```
{
  "current_units": { "temperature_2m": "°C", "wind_speed_10m": "km/h" },
  "current": {
    "time": "2026-10-09T09:30",
```

```
    "temperature_2m": 22.4,
    "weather_code": 3,
    "wind_speed_10m": 9.1
  }
}
```

5.2 Geocoding (nome → coordenadas)

```
GET https://geocoding-api.open-meteo.com/v1/search
    ?name=S%C3%A3o%20Paulo          ← encodeURIComponent("São Paulo")
    &count=1&language=pt&format=json
```

```
{ "results": [ { "name": "São Paulo", "latitude": -23.5475, "longitude": -46.6361, "country": "Brasil" } ] }

// Cidade inexistente → o campo "results" SOME (não vem array vazio):
{ "generationtime_ms": 0.4 }
```

> **Confira na fonte**
> Os exemplos acima mostram o **formato**; os valores reais mudam. Teste no navegador (cole a URL na barra de endereços) e leia a documentação em `open-meteo.com`. A fonte da verdade é a API e a documentação — não o palpite da IA.

5.3 Códigos de tempo (weather_code, padrão WMO) usados no projeto

Código	Texto	Ícone	Código	Texto	Ícone
0	Céu limpo		61	Chuva fraca	
1	Predominantemente limpo		63	Chuva moderada	
2	Parcialmente nublado		65	Chuva forte	
3	Nublado		80	Pancadas de chuva fracas	
45	Nevoeiro		81 / 82	Pancadas moderadas / fortes	 / 
51 / 53 / 55	Garoa fraca / moderada / intensa		95	Trovoada	
Qualquer outro código → `{ texto: 'Condição desconhecida', icone: '❓' }` (nunca deixar quebrar)					

6. Fluxos principais

6.1 Ver o clima de uma cidade

```
clique em "Ver clima" ──► selecionarCidade(cidade)
      └─► carregarClima(cidade, forcar=false)
            ├─ cache fresco (< 10 min)?  ──► atualizarTela()              (sem rede)
            └─ não: mostrarStatus('Carregando…','carregando')
                    try { buscarClima() → salvarNoCache() → mostrarStatus('…atualizado','sucesso') → atualizarTela() }
                    catch { há cache antigo? → mostra com aviso "Sem conexão…"
                            senão → mostrarStatus('Não foi possível carregar…','erro') }
```

6.2 Buscar por nome (⭐)

```
submit do formulário ──► preventDefault() ──► nome vazio? → erro + foco no campo
      └─► buscarCoordenadas(nome)            (Geocoding)
            ├─ null → "Cidade não encontrada" (erro)
            └─ { nome, latitude, longitude } ──► adicionarCidade() ──► carregarClima()
```

6.3 Atualização automática (⭐)

```
marcar "Atualizar a cada 10 minutos" ──► iniciarAtualizacao()
      ├─ pararAtualizacao()                      ← nunca 2 timers vivos
      └─ timerAtualizacao = setInterval(atualizarTudo, 10 * MINUTO)
aba oculta (visibilitychange) ──► pararAtualizacao()      aba visível ──► atualizarTudo() + iniciarAtualizacao()
```

7. Tratamento do "caminho triste"

O que pode dar errado	Como acontece	Como tratar
1ª visita: chave inexistente	`getItem` → `null` ; `JSON.parse(null)` → `null` ; `forEach` quebra	`JSON.parse(...) || []`
JSON corrompido no storage	`JSON.parse` lança `SyntaxError`	`try/catch` devolvendo o valor padrão
Cota cheia ou storage bloqueado	`setItem` lança erro	`try/catch` e retorno `true/false`
Rede caída	O `fetch` rejeita	`try/catch` + mensagem acessível
Servidor responde 404/500	O `fetch` **não** rejeita; `response.ok` é `false`	`if (!resposta.ok) throw new Error(...)`
Servidor lento	"Carregando…" para sempre	`AbortController` + `setTimeout` de 8 s; `AbortError` vira "demorou demais"
Cidade inexistente	Geocoding sem o campo `results`	`if (!dados.results || dados.results.length === 0) return null`
Código de tempo desconhecido	`mapa[code]` é `undefined`	`mapa[code] || { texto: 'Condição desconhecida', icone: '❓' }`

Geolocalização negada/indisponível	Callback de erro; ou `navigator.geolocation` inexistente	Avisar e focar o campo de busca (fallback)
Timers acumulados	`setInterval` criado várias vezes sem guardar o id	Guardar o id; `clearInterval` antes de criar outro

8. Modelos de código (arquivos da pasta `modelo-de-arquivos`)

Abaixo, o conteúdo integral dos arquivos-modelo. **HTML e CSS** já estão prontos para você ler, entender e adaptar. Nos arquivos **JavaScript**, procure os `TODO`: cada um é uma função para você escrever. Funções já prontas servem de exemplo de estilo.

index.html

Estrutura semântica pronta: landmarks, skip link, formulário com `label for`, região `aria-live`, card com `dl` / `time`, `aside` da atualização automática e os scripts na ordem certa.

```
<!DOCTYPE html>
<!-- lang="pt-BR": diz ao leitor de tela e ao buscador qual é o idioma da página -->
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />

  <!-- SEO on-page: título da aba/resultado do Google + resumo exibido na busca -->
  <title>PrevisaoCerta — Painel climático da AgroLog</title>
  <meta name="description" content="Painel operacional da AgroLog para acompanhar o clima nas cidades de coleta e entrega."
/>

  <!-- Open Graph: prévia ao compartilhar o link -->
  <meta property="og:title" content="PrevisaoCerta — AgroLog" />
  <meta property="og:description" content="Clima das cidades de coleta e entrega da AgroLog." />

  <link rel="icon" href="img/favicon.svg" type="image/svg+xml" />
  <link rel="stylesheet" href="css/style.css" />
</head>
<body>
  <!-- Skip link: primeiro elemento focável da página (teclado) -->
  <a class="pular" href="#conteudo">Pular para o conteúdo</a>

  <!-- LANDMARK 1: header -->
  <header class="cabecalho">
    <div class="container cabecalho-interno">
      <div class="marca">
        <!-- Logo decorativo: alt="" (o nome AgroLog já está escrito ao lado) -->
        <img src="img/logo-agrolog.svg" alt="" width="48" height="48" />
        <div>
          <p class="marca-nome">AgroLog · PrevisaoCerta</p>
          <!-- UM só h1 na página -->
          <h1>Clima das cidades de coleta e entrega</h1>
        </div>
      </div>

      <!-- LANDMARK 2: nav (com rótulo, para diferenciar de outras navegações) -->
      <nav aria-label="Navegação principal">
        <div class="menu">
          <a href="#buscar">Buscar</a>
          <a href="#monitoradas">Cidades</a>
          <a href="#detalhe">Clima atual</a>
        </div>
      </nav>
    </div>
  </header>

  <!-- LANDMARK 3: main (UM só por página; tabindex="-1" deixa o skip link mover o foco para cá) -->
  <main id="conteudo" class="container" tabindex="-1">

    <!-- Região viva: o leitor de tela ANUNCIA o que o JavaScript escrever aqui.
         Precisa existir no HTML ANTES da mudança. [hidden] = vazia/escondida. -->
```

```
    <p id="status" class="status" role="status" aria-live="polite" hidden></p>

    <div class="layout">
      <div>
        <!-- SEÇÃO: buscar cidade (h2 liga o título à seção por aria-labelledby) -->
        <section id="buscar" class="secao" aria-labelledby="t-buscar">
          <h2 id="t-buscar">Buscar cidade</h2>

          <!-- role="search" marca o formulário como busca (landmark) -->
          <form id="form-busca" class="form-busca" role="search">
            <!-- label for="..." liga o rótulo ao campo pelo id -->
            <label for="input-cidade">Nome da cidade</label>
            <div class="campo-linha">
              <input type="search" id="input-cidade" name="cidade" autocomplete="off" required />
              <button type="submit" id="btn-buscar" class="btn">Buscar</button>
            </div>
          </form>

          <button type="button" id="btn-localizacao" class="btn btn-secundario" style="margin-top:12px">
            Usar minha localização
          </button>

          <!-- Sugestões com coordenadas fixas (caminho-base: sem Geocoding) -->
          <div class="sugestoes" role="group" aria-label="Cidades sugeridas">
            <button type="button" class="btn btn-secundario btn-sugestao"
                    data-nome="Florianópolis" data-lat="-27.5954" data-lon="-48.5480">Florianópolis</button>
            <button type="button" class="btn btn-secundario btn-sugestao"
                    data-nome="Curitiba" data-lat="-25.4284" data-lon="-49.2733">Curitiba</button>
            <button type="button" class="btn btn-secundario btn-sugestao"
                    data-nome="Porto Alegre" data-lat="-30.0346" data-lon="-51.2177">Porto Alegre</button>
            <button type="button" class="btn btn-secundario btn-sugestao"
                    data-nome="Chapecó" data-lat="-27.1004" data-lon="-52.6152">Chapecó</button>
          </div>

          <!-- Preferência °C/°F (persistida no localStorage) -->
          <fieldset>
            <legend>Unidade de temperatura</legend>
            <label class="opcao"><input type="radio" name="unidade" value="celsius" checked /> °C</label>
            <label class="opcao"><input type="radio" name="unidade" value="fahrenheit" /> °F</label>
          </fieldset>
        </section>

        <!-- SEÇÃO: cidades monitoradas -->
        <section id="monitoradas" class="secao" aria-labelledby="t-monitoradas">
          <h2 id="t-monitoradas">Cidades monitoradas</h2>
          <p id="msg-vazio" class="vazio">Nenhuma cidade monitorada ainda. Busque uma cidade ou use as sugestões.</p>
          <!-- Lista de verdade: o leitor de tela anuncia "lista com N itens" -->
          <ul id="lista-cidades" class="lista-cidades"></ul>
          <p style="margin-top:12px">
            <button type="button" id="btn-limpar" class="btn btn-perigo">Limpar tudo</button>
          </p>
        </section>
      </div>

      <div>
        <!-- ARTICLE: conteúdo independente (o card faz sentido sozinho) -->
        <section id="detalhe" class="secao" aria-labelledby="t-detalhe">
          <h2 id="t-detalhe">Clima atual</h2>
          <article id="card-clima" class="card-clima">
            <h3 class="card-cidade">Selecione uma cidade</h3>
            <dl>
              <div>
                <dt>Temperatura</dt>
                <dd class="card-temp">—</dd>
              </div>
              <div>
                <dt>Condição</dt>
```

```
                <dd class="card-desc">—</dd>
              </div>
              <div>
                <dt>Atualizado às</dt>
                <dd><time class="card-hora">—</time></dd>
              </div>
            </dl>
          </article>
        </section>

        <!-- ASIDE: complemento (atualização automática com setInterval) -->
        <aside class="secao" aria-labelledby="t-auto">
          <h2 id="t-auto">Atualização automática</h2>
          <label class="auto-linha" for="chk-auto">
            <input type="checkbox" id="chk-auto" />
            Atualizar a cada 10 minutos
          </label>
          <p id="ultima-atualizacao" class="nota">Ainda não atualizado.</p>
        </aside>
      </div>
    </div>
  </main>

  <!-- LANDMARK 4: footer -->
  <footer class="rodape">
    <div class="container">
      <p>PrevisaoCerta — painel climático operacional da AgroLog. Dados: <a href="https://open-meteo.com/">Open-Meteo</a>.
</p>
    </div>
  </footer>

  <!-- Ordem importa: cada arquivo usa funções do anterior -->
  <script src="js/storage.js"></script>
  <script src="js/api.js"></script>
  <script src="js/ui.js"></script>
  <script src="js/app.js"></script>
</body>
</html>
```

css/style.css

Tokens da identidade visual (`:root`), reset, foco visível, skip link, componentes, estados de status, Grid responsivo e media queries mobile-first.

```
/* ========================================================================
   PrevisaoCerta — AgroLog | Folha de estilo (GABARITO)
   Mobile-first: o CSS base vale para o celular; media queries ACRESCENTAM.
   ======================================================================== */

/* 1. TOKENS DA IDENTIDADE VISUAL ------------------------------------------
   Todas as cores moram aqui. Contrastes (WCAG AA exige 4,5:1 para texto):
   branco sobre --verde-folha = 5,1:1 · branco sobre --verde-campo = 7,9:1
   grafite sobre --fundo = ~15:1 · --texto-suave sobre --fundo = 5,2:1       */
:root {
  --verde-campo: #1b5e20;   /* cabeçalho, títulos */
  --verde-folha: #2e7d32;   /* botões principais */
  --verde-claro: #e8f5e9;   /* superfícies de destaque */
  --ambar-foco: #ffb300;    /* anel de foco e detalhes (nunca como cor de texto) */
  --grafite: #212121;       /* texto principal */
  --texto-suave: #5f6b61;   /* texto secundário */
  --fundo: #f4f7f2;         /* fundo da página */
  --superficie: #ffffff;    /* cartões */
  --borda: #cfd8cc;
  --erro: #b71c1c;
  --erro-fundo: #fdecea;
```

```
  --info: #0d47a1;
  --info-fundo: #e3f0fd;
  --raio: 10px;
  --espaco: 16px;
  --fonte: "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif;
}

/* 2. RESET E BASE ------------------------------------------------------- */
*, *::before, *::after { box-sizing: border-box; }

body {
  margin: 0;
  font-family: var(--fonte);
  color: var(--grafite);
  background: var(--fundo);
  line-height: 1.5;
}

img { max-width: 100%; height: auto; }

.container {
  width: 100%;
  max-width: 1000px;
  margin: 0 auto;
  padding: 0 var(--espaco);
}

h1, h2, h3 { line-height: 1.2; margin: 0 0 8px; }
h1 { font-size: clamp(1.4rem, 5vw, 2rem); }
h2 { font-size: 1.3rem; color: var(--verde-campo); }
h3 { font-size: 1.1rem; }

/* 3. ACESSIBILIDADE ----------------------------------------------------
   NUNCA remover o outline: quem navega por teclado precisa ver o foco.        */
a:focus-visible,
button:focus-visible,
input:focus-visible,
select:focus-visible {
  outline: 3px solid var(--ambar-foco);
  outline-offset: 2px;
}

/* Skip link: invisível até receber foco pelo teclado */
.pular {
  position: absolute;
  left: -999px;
  top: 8px;
  background: var(--grafite);
  color: #fff;
  padding: 8px 14px;
  border-radius: var(--raio);
  z-index: 10;
}
.pular:focus { left: 8px; }

/* Texto só para leitor de tela */
.so-leitor {
  position: absolute;
  width: 1px; height: 1px;
  overflow: hidden;
  clip: rect(0 0 0 0);
  white-space: nowrap;
}

/* 4. CABEÇALHO E RODAPÉ ------------------------------------------------- */
.cabecalho {
  background: var(--verde-campo);
```

```
  color: #fff;
  padding: 14px 0;
}
.cabecalho-interno {
  display: flex;
  flex-direction: column;       /* celular: empilhado */
  gap: 12px;
}
.marca { display: flex; align-items: center; gap: 12px; }
.marca-nome { margin: 0; font-size: 0.85rem; letter-spacing: 0.08em; text-transform: uppercase; }
.cabecalho h1 { margin: 0; }

.menu { display: flex; gap: 16px; flex-wrap: wrap; }
.menu a { color: #fff; padding: 6px 2px; }

.rodape {
  margin-top: 32px;
  padding: 20px 0;
  background: var(--grafite);
  color: #e0e0e0;
  font-size: 0.9rem;
}

/* 5. CARTÕES E FORMULÁRIOS ------------------------------------------------- */
main { padding-top: 20px; }

.secao {
  background: var(--superficie);
  border: 1px solid var(--borda);
  border-radius: var(--raio);
  padding: var(--espaco);
  margin-bottom: var(--espaco);
}

.form-busca { display: flex; flex-direction: column; gap: 8px; }
.form-busca label { font-weight: 600; }
.campo-linha { display: flex; flex-direction: column; gap: 8px; }

input[type="search"] {
  font: inherit;
  padding: 10px 12px;
  border: 2px solid var(--borda);
  border-radius: var(--raio);
  min-height: 44px;                 /* alvo de toque confortável */
}

.btn {
  font: inherit;
  font-weight: 600;
  min-height: 44px;
  padding: 10px 16px;
  border: 2px solid var(--verde-folha);
  border-radius: var(--raio);
  background: var(--verde-folha);
  color: #fff;
  cursor: pointer;
  transition: background 0.2s, transform 0.1s;   /* transition no estado BASE */
}
.btn:hover { background: var(--verde-campo); }
.btn:active { transform: scale(0.98); }
.btn-secundario { background: var(--superficie); color: var(--verde-campo); }
.btn-secundario:hover { background: var(--verde-claro); }
.btn-perigo { border-color: var(--erro); background: var(--superficie); color: var(--erro); }
.btn-perigo:hover { background: var(--erro-fundo); }

fieldset { border: 1px solid var(--borda); border-radius: var(--raio); padding: 8px 12px; margin: 12px 0 0; }
legend { font-weight: 600; padding: 0 6px; }
```

```css
.opcao { display: inline-flex; align-items: center; gap: 6px; margin-right: 16px; min-height: 44px; }

.sugestoes { display: flex; flex-wrap: wrap; gap: 8px; margin-top: 12px; }

/* 6. STATUS: os 3 estados (carregando / sucesso / erro) ----------------------- */
.status {
  margin: 0 0 var(--espaco);
  padding: 12px 14px;
  border-radius: var(--raio);
  border-left: 6px solid var(--info);
  background: var(--info-fundo);
  color: var(--info);
}
.status[hidden] { display: none; }
.status.sucesso { border-color: var(--verde-folha); background: var(--verde-claro); color: var(--verde-campo); }
.status.erro { border-color: var(--erro); background: var(--erro-fundo); color: var(--erro); }
/* cor + borda + texto: a informação nunca depende só da cor */

/* 7. LISTA DE CIDADES ---------------------------------------------------------- */
.lista-cidades {
  list-style: none;
  margin: 0;
  padding: 0;
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));   /* Grid responsivo sem media query */
  gap: 12px;
}
.cidade-item {
  display: flex;
  flex-direction: column;
  gap: 8px;
  padding: 12px;
  border: 1px solid var(--borda);
  border-radius: var(--raio);
  background: var(--fundo);
}
.cidade-item.ativa { border-color: var(--verde-folha); background: var(--verde-claro); }
.cidade-nome { font-weight: 700; }
.cidade-mini { color: var(--texto-suave); }
.cidade-acoes { display: flex; gap: 8px; flex-wrap: wrap; }
.vazio { color: var(--texto-suave); margin: 0; }

/* 8. CARD DE CLIMA ------------------------------------------------------------- */
.card-clima dl { margin: 0; display: grid; grid-template-columns: 1fr; gap: 4px 16px; }
.card-clima dt { color: var(--texto-suave); font-size: 0.85rem; }
.card-clima dd { margin: 0 0 8px; font-size: 1.1rem; font-weight: 600; }
.card-temp { font-size: 2.4rem; font-weight: 700; color: var(--verde-campo); }
.card-icone { font-size: 2rem; }

.auto-linha { display: flex; align-items: center; gap: 8px; min-height: 44px; }
.nota { color: var(--texto-suave); font-size: 0.9rem; margin: 4px 0 0; }

/* 9. TABLET E DESKTOP: ACRESCENTA (mobile-first) ------------------------------- */
@media (min-width: 48em) {                      /* 768px */
  .cabecalho-interno { flex-direction: row; align-items: center; justify-content: space-between; }
  .campo-linha { flex-direction: row; align-items: stretch; }
  .campo-linha input { flex: 1; }
  .card-clima dl { grid-template-columns: repeat(3, 1fr); }
}

@media (min-width: 64em) {                      /* 1024px */
  .layout { display: grid; grid-template-columns: 2fr 1fr; gap: var(--espaco); align-items: start; }
}

/* 10. PREFERÊNCIAS DO USUÁRIO ------------------------------------------------- */
@media (prefers-reduced-motion: reduce) {
```

```
  * { transition: none !important; animation: none !important; }
}
```

js/storage.js

A "memória" do painel. Escreva as funções marcadas com `TODO` (cidades, unidade, cache). Dica: `JSON.parse(...) || []` e `try/catch`.

```
/* ========================================================================
   storage.js — a "memória" do painel (localStorage)  | Aula de quinta (08/10)
   Regras de ouro:
   1) o localStorage só guarda STRING → JSON.stringify ao salvar, JSON.parse ao ler
   2) na 1ª visita a chave não existe → getItem devolve null → use "|| []"
   3) nunca guardar token, senha ou CPF aqui (qualquer um lê no DevTools)
   ======================================================================== */

// Prefixo "previsaocerta:" evita colisão com outras chaves do mesmo site
const CHAVE_CIDADES = 'previsaocerta:cidades';
const CHAVE_UNIDADE = 'previsaocerta:unidade';
const CHAVE_CACHE = 'previsaocerta:cache';
const CHAVE_AUTO = 'previsaocerta:auto';

/* ---------- Cidades monitoradas (array de objetos) ---------- */

function carregarCidades() {
  // TODO: ler a chave CHAVE_CIDADES com getItem, converter com JSON.parse e devolver [] na 1ª visita (JSON.parse(null) é
null!). Use try/catch para JSON corrompido.
}

function salvarCidades(cidades) {
  // TODO: gravar com setItem + JSON.stringify; devolver true/false (try/catch: cota cheia ou armazenamento bloqueado).
}

// Devolve false se a cidade já estava na lista (evita duplicata)
function adicionarCidade(cidade) {
  // TODO: carregar a lista, impedir duplicata (nome sem diferenciar maiúsculas) com some(), dar push e salvar. Devolva
false se já existia.
}

function removerCidade(nome) {
  // TODO: usar filter para manter todas as cidades menos a removida e salvar o resultado.
}

function limparTudo() {
  // TODO: removeItem das chaves de cidades e de cache (NÃO use localStorage.clear()).
}

/* ---------- Preferência de unidade (string simples) ---------- */

function carregarUnidade() {
  // TODO: devolver o valor salvo em CHAVE_UNIDADE ou 'celsius' quando não houver nada (||).
}

function salvarUnidade(unidade) {
  // TODO: gravar a unidade com setItem (dentro de try/catch).
}

/* ---------- Atualização automática ligada/desligada ---------- */

function carregarAuto() {
  try {
    return localStorage.getItem(CHAVE_AUTO) === 'true';
  } catch (erro) {
    return false;
  }
```

```
}

function salvarAuto(ligado) {
  try {
    localStorage.setItem(CHAVE_AUTO, String(ligado));
  } catch (erro) {
    /* ignora */
  }
}

/* ---------- Cache da última resposta da API (objeto indexado por cidade) ---------- */

function lerCache() {
  // TODO: ler o objeto de cache (JSON.parse(...) || {}) com try/catch.
}

function salvarNoCache(nome, dado) {
  // TODO: guardar o dado indexado pelo nome da cidade com o carimbo salvoEm: Date.now().
}

// Fresco = salvo há menos de "limiteMs" milissegundos
function cacheFresco(entrada, limiteMs) {
  return Boolean(entrada) && Date.now() - entrada.salvoEm < limiteMs;
}
```

js/api.js

A "internet" do painel. Complete `buscarJSON` (tempo limite + `response.ok`), `buscarCoordenadas`, `buscarClima` e `decodificarTempo`. `formatarTemperatura` já vem pronta.

```
/* ========================================================================
   api.js — conversa com a internet (fetch)  | Aula de sexta (09/10)
   API: Open-Meteo (pública, sem chave) — https://open-meteo.com/
   Duas chamadas encadeadas: Geocoding (nome → lat/long) → Forecast (lat/long → clima)
   ======================================================================== */

const URL_GEOCODING = 'https://geocoding-api.open-meteo.com/v1/search';
const URL_FORECAST = 'https://api.open-meteo.com/v1/forecast';
const LIMITE_MS = 8000;   // tempo máximo de espera por requisição (8 s)

/* Faz um fetch com TEMPO LIMITE: sem isso, rede lenta = "Carregando…" para sempre.
   AbortController cancela o fetch; setTimeout dispara o cancelamento. */
async function buscarJSON(url) {
  // TODO: criar AbortController + setTimeout(LIMITE_MS) para abortar; await fetch(url, { signal }); se !resposta.ok lançar
Error; devolver await resposta.json(); clearTimeout no finally.
}

/* ⭐ Geocoding: "Florianópolis" → { nome, latitude, longitude } (ou null) */
async function buscarCoordenadas(nome) {
  // TODO: montar a URL do Geocoding com encodeURIComponent(nome); se não existir dados.results (ou vier vazio) devolver
null; senão devolver { nome, latitude, longitude } do 1º resultado.
}

/* Forecast: lat/long → clima atual { temperatura (°C), codigo, vento, hora } */
async function buscarClima(latitude, longitude) {
  // TODO: montar a URL do Forecast (current=temperature_2m,weather_code,wind_speed_10m&timezone=auto) e devolver {
temperatura, codigo, vento, hora } a partir de dados.current.
}

/* weather_code (WMO) → texto em pt-BR + emoji */
function decodificarTempo(code) {
  // TODO: mapa { código: { texto, icone } } com os códigos WMO do documento funcional; devolver mapa[code] || { texto:
'Condição desconhecida', icone: '❓' }.
}
```

```
/* A API devolve °C; a conversão para °F é feita aqui (sem novo fetch) */
function formatarTemperatura(celsius, unidade) {
  if (unidade === 'fahrenheit') {
    return Math.round(celsius * 9 / 5 + 32) + '°F';
  }
  return Math.round(celsius) + '°C';
}
```

js/ui.js

A "tela" do painel. Complete `mostrarAviso` (`setTimeout`), `renderizarLista` e `renderizarCard` . As funções de status e `limparCard` já vêm prontas.

```
/* ========================================================================
   ui.js — desenha na tela (DOM)  | usa createElement, classList, textContent (S08)
   Nada aqui busca dados nem salva: só recebe informação e mostra.
   ======================================================================== */

const elStatus = document.getElementById('status');
let timerAviso = null;   // id do setTimeout do aviso (para poder cancelar)

/* ---------- Os 3 estados: carregando / sucesso / erro ---------- */

// tipo: 'carregando' | 'sucesso' | 'erro'
function mostrarStatus(texto, tipo) {
  clearTimeout(timerAviso);                // cancela um aviso anterior que ainda ia sumir
  elStatus.className = 'status ' + tipo;
  elStatus.textContent = texto;            // a região aria-live anuncia a mudança
  elStatus.hidden = false;
}

function esconderStatus() {
  clearTimeout(timerAviso);
  elStatus.hidden = true;
  elStatus.textContent = '';
}

/* Aviso de sucesso que SOME sozinho depois de 3 s (setTimeout).
   Erros NÃO usam esta função: mensagem de erro não deve desaparecer sozinha. */
function mostrarAviso(texto) {
  // TODO: mostrar o texto com mostrarStatus(texto, "sucesso") e agendar com setTimeout (3000 ms) a chamada de
esconderStatus; guardar o id em timerAviso.
}

/* ---------- Lista de cidades ---------- */

function formatarHora(iso) {
  // "2026-10-09T09:30" → "09:30" (a API já devolve a hora local da cidade)
  return iso ? iso.slice(11, 16) : '—';
}

/* Redesenha a <ul> inteira a partir do array (a "verdade" está no localStorage). */
function renderizarLista(cidades, ativa, unidade, cache, aoSelecionar, aoRemover) {
  // TODO: limpar a <ul>; para cada cidade criar <li> com nome, resumo do cache, botão "Ver clima" e botão "Remover" (com
aria-label); use createElement/classList/textContent/addEventListener.
}

/* ---------- Card de clima ---------- */

function renderizarCard(dado, unidade) {
  // TODO: preencher .card-cidade, .card-temp, .card-desc e o <time> (textContent + datetime) a partir de dado e unidade.
}

function limparCard() {
  const card = document.getElementById('card-clima');
```

```
  card.querySelector('.card-cidade').textContent = 'Selecione uma cidade';
  card.querySelector('.card-temp').textContent = '—';
  card.querySelector('.card-desc').textContent = '—';
  const hora = card.querySelector('.card-hora');
  hora.textContent = '—';
  hora.removeAttribute('datetime');
}
```

js/app.js

O "maestro". Complete `carregarClima`, `usarMinhaLocalizacao`, `atualizarTudo` e `iniciarAtualizacao`. Eventos, `visibilitychange` e a inicialização já vêm prontos.

```
/* ========================================================================
   app.js — o "maestro": liga eventos, storage, API e tela
   Fluxo: evento → (storage/api) → renderizar
   ======================================================================== */

const MINUTO = 60 * 1000;
const VALIDADE_CACHE = 10 * MINUTO;   // dado com menos de 10 min é considerado fresco

let cidadeAtual = null;               // objeto { nome, latitude, longitude } selecionado
let unidade = carregarUnidade();      // 'celsius' | 'fahrenheit' (vem do localStorage)
let timerAtualizacao = null;          // id do setInterval (nunca criar 2!)

/* ------------------------------------------------------------------
   Mensagem amigável a partir do erro (o usuário não precisa ver stack)
   ------------------------------------------------------------------ */
function mensagemDeErro(erro) {
  if (erro && erro.name === 'AbortError') {
    return 'A busca demorou demais. Tente de novo.';
  }
  return 'Não foi possível carregar. Tente de novo.';
}

/* ------------------------------------------------------------------
   Redesenha lista + card a partir do que está salvo
   ------------------------------------------------------------------ */
function atualizarTela() {
  const cidades = carregarCidades();
  const cache = lerCache();
  renderizarLista(
    cidades,
    cidadeAtual && cidadeAtual.nome,
    unidade,
    cache,
    selecionarCidade,
    excluirCidade
  );

  const dado = cidadeAtual && cache[cidadeAtual.nome];
  if (dado) {
    renderizarCard({ nome: cidadeAtual.nome, ...dado }, unidade);
  } else {
    limparCard();
  }
}

/* ------------------------------------------------------------------
   Busca o clima de UMA cidade com os 3 estados + cache + fallback offline
   ------------------------------------------------------------------ */
async function carregarClima(cidade, forcar) {
  // TODO: usar cache fresco quando possível; senão mostrar "Carregando…", buscarClima, salvarNoCache e "sucesso"; no catch
mostrar erro amigável (ou o dado em cache com aviso de "sem conexão").
}
```

```
/* ----------------------------------------------------------------
   Ações do usuário
   ---------------------------------------------------------------- */
function selecionarCidade(cidade) {
  cidadeAtual = cidade;
  carregarClima(cidade, false);
}

function excluirCidade(cidade) {
  removerCidade(cidade.nome);
  if (cidadeAtual && cidadeAtual.nome === cidade.nome) {
    cidadeAtual = null;
  }
  atualizarTela();
  mostrarAviso(cidade.nome + ' removida.');
}

// Adiciona à lista e já mostra o clima
function adicionarEMostrar(cidade) {
  const salvou = adicionarCidade(cidade);
  cidadeAtual = cidade;
  if (salvou) {
    atualizarTela();                              // mostra a cidade na lista já
  }
  carregarClima(cidade, false);
}

/* ⭐ Busca por nome: Geocoding → Forecast */
async function aoBuscar(evento) {
  evento.preventDefault();                        // não recarregar a página
  const campo = document.getElementById('input-cidade');
  const nome = campo.value.trim();
  if (nome === '') {
    mostrarStatus('Digite o nome de uma cidade.', 'erro');
    campo.focus();
    return;
  }

  mostrarStatus('Procurando "' + nome + '"…', 'carregando');
  try {
    const cidade = await buscarCoordenadas(nome);
    if (cidade === null) {
      mostrarStatus('Cidade "' + nome + '" não encontrada. Confira a grafia.', 'erro');
      campo.focus();
      return;
    }
    campo.value = '';
    adicionarEMostrar(cidade);
  } catch (erro) {
    mostrarStatus(mensagemDeErro(erro), 'erro');
  }
}

/* Browser API: Geolocation (pede permissão; só funciona em contexto seguro) */
function usarMinhaLocalizacao() {
  // TODO: verificar navigator.geolocation; getCurrentPosition(sucesso, erro, { timeout: 10000 }); no erro avisar e focar o
campo de busca (fallback).
}

/* ----------------------------------------------------------------
   Atualização automática (setInterval) — UM timer só
   ---------------------------------------------------------------- */
async function atualizarTudo() {
  // TODO: buscar o clima de TODAS as cidades em paralelo (Promise.all), cada uma com seu try/catch; salvar no cache,
redesenhar e escrever a hora da última atualização.
}
```

```
function pararAtualizacao() {
  clearInterval(timerAtualizacao);
  timerAtualizacao = null;
}

function iniciarAtualizacao() {
  // TODO: chamar pararAtualizacao() antes (nunca 2 timers) e guardar em timerAtualizacao o setInterval(atualizarTudo, 10 *
MINUTO).
}

/* ----------------------------------------------------------------
   Inicialização
   ---------------------------------------------------------------- */
function iniciar() {
  // Unidade salva → marcar o rádio certo
  document.querySelector('input[name="unidade"][value="' + unidade + '"]').checked = true;

  // Eventos
  document.getElementById('form-busca').addEventListener('submit', aoBuscar);
  document.getElementById('btn-localizacao').addEventListener('click', usarMinhaLocalizacao);

  document.querySelectorAll('.btn-sugestao').forEach(function (botao) {
    botao.addEventListener('click', function () {
      adicionarEMostrar({
        nome: botao.dataset.nome,
        latitude: Number(botao.dataset.lat),        // dataset sempre devolve string
        longitude: Number(botao.dataset.lon)
      });
    });
  });

  document.querySelectorAll('input[name="unidade"]').forEach(function (radio) {
    radio.addEventListener('change', function () {
      unidade = radio.value;
      salvarUnidade(unidade);                       // lembra após o F5
      atualizarTela();                              // sem novo fetch: só reformata
    });
  });

  document.getElementById('btn-limpar').addEventListener('click', function () {
    limparTudo();
    cidadeAtual = null;
    atualizarTela();
    mostrarAviso('Lista limpa.');
  });

  const chkAuto = document.getElementById('chk-auto');
  chkAuto.checked = carregarAuto();
  chkAuto.addEventListener('change', function () {
    salvarAuto(chkAuto.checked);
    if (chkAuto.checked) {
      iniciarAtualizacao();
    } else {
      pararAtualizacao();
    }
  });
  if (chkAuto.checked) {
    iniciarAtualizacao();
  }

  // Aba escondida → pausa o timer; ao voltar, atualiza na hora e recomeça
  document.addEventListener('visibilitychange', function () {
    if (!chkAuto.checked) {
      return;
    }
    if (document.hidden) {
```

```
      pararAtualizacao();
    } else {
      atualizarTudo();
      iniciarAtualizacao();
    }
  });

  // Primeira pintura a partir do que estava salvo (sobrevive ao F5)
  const cidades = carregarCidades();
  if (cidades.length > 0) {
    cidadeAtual = cidades[0];
  }
  atualizarTela();
  if (cidadeAtual) {
    carregarClima(cidadeAtual, false);
  }
}

iniciar();
```

9. Passo a passo sugerido

Etapa	O que fazer	Como saber que funcionou
1 (terça)	Abrir o `index.html` e o `style.css` do modelo; ler cada comentário; trocar nomes, textos e cores mantendo os tokens; rodar o Lighthouse	Acessibilidade e SEO ≥ 90; Tab percorre tudo na ordem certa
2 (quinta)	Escrever `storage.js`: `carregarCidades` (`|| []`), `salvarCidades` (`stringify`), `adicionarCidade`, `removerCidade`, `limparTudo`, `carregarUnidade` / `salvarUnidade`	No Console: `carregarCidades()` devolve `[]` na 1ª visita; a chave aparece em Application
3 (quinta)	Escrever `renderizarLista` e `mostrarAviso` em `ui.js`	A lista reaparece após F5; o aviso some em 3 s
4 (sexta)	Escrever `buscarJSON` e `buscarClima`; depois `decodificarTempo` e `renderizarCard`	Aba Network mostra o pedido (200); o card mostra o clima
5 (sexta)	Completar `carregarClima` com os 3 estados e o cache	Network › Offline: aparece a mensagem de erro, não tela branca
6 (sexta ⭐)	`buscarCoordenadas` e `usarMinhaLocalizacao`	Cidade inexistente não gera erro no Console; localização negada leva à busca por nome
7 (sexta ⭐)	`atualizarTudo` e `iniciarAtualizacao`	"Timers"/Network: nunca dois pedidos por ciclo

10. Plano de testes (cenários)

#	Cenário	Resultado esperado
T1	Abrir com o `localStorage` vazio (Application › Clear site data)	Mensagem de lista vazia; **nenhum** erro no Console
T2	Adicionar 3 cidades, F5, fechar e reabrir a aba	Lista e unidade preservadas
T3	Escolher °F e F5	°F continua marcado; valores reformatados sem nova requisição
T4	Network › Offline e "Ver clima"	Mensagem de erro anunciada; sem tela em branco
T5	Trocar a URL do Forecast por um endereço inexistente (404)	Mensagem de erro (não "undefined°C")
T6	Buscar "Atlantida de Marte" e "São Paulo"	"não encontrada" sem erro no Console; acento e espaço funcionam
T7	Usar minha localização: permitir e bloquear	Permitir: cidade adicionada; bloquear: aviso + foco no campo
T8	Ligar a atualização automática, trocar de aba e voltar	Um pedido por ciclo; pausa ao esconder; atualiza ao voltar
T9	Navegar só com Tab/Shift+Tab/Enter; ligar o Narrador	Foco sempre visível; nomes e mensagens anunciados
T10	Rodar o Lighthouse (Acessibilidade e SEO) e testar em 360px, 768px e 1280px	Notas ≥ 90; sem rolagem horizontal

11. Armadilhas frequentes (e dicas para revisar código de IA)

- **Um só `await`:** sem o segundo (`await resposta.json()`) você recebe uma Promise, não os dados.
- **Achar que 404 cai no `catch`:** não cai; use `response.ok`.
- **Esquecer `encodeURIComponent`:** "São Paulo" quebra a URL.
- **`results[0]` sem checar:** cidade inexistente gera `TypeError`.
- **`setInterval` dentro de um clique:** cada clique cria um timer novo; guarde o id e pare o anterior.
- **Tempos em segundos:** `setTimeout` e `setInterval` usam **milissegundos**.
- **`innerHTML` com dado de fora:** use `textContent`.
- **Mensagem de erro que some sozinha ou só muda de cor:** falha de acessibilidade.
- **Código da IA:** a IA entrega o "caminho feliz". Pergunte-se: e na 1ª visita? e sem rede? e com 404? e se negarem a permissão? e se eu clicar 5 vezes? E só use o que você consegue explicar.

12. Checklist final de entrega

- ✓ Um só `h1`; landmarks corretos; `lang="pt-BR"`; `title` e `meta description`.
- ✓ Todo campo com `label for`; `alt` decorativo e informativo corretos; foco visível; skip link.
- ✓ Cidades e unidade sobrevivem ao F5; sem erro de `null` na 1ª visita.
- ✓ `fetch` com dois `await`, `response.ok`, `try/catch` e os 3 estados.
- ✓ Rede offline mostra mensagem acessível, não tela branca.
- ✓ ⭐ Geocoding com `results` checado; geolocalização com fallback; um só intervalo ativo; aviso com `setTimeout`.
- ✓ Lighthouse de Acessibilidade e SEO ≥ 90 (com prints antes e depois).
- ✓ Você explica cada função, inclusive o que a IA errou e como consertou.
